

Layers of a Persistent World

A Graded Account of Rendering, State, Provenance, and Committed History

Flyxion
Independent Researcher

September 2026

Abstract

Research on persistent multiplayer worlds, learned world models, distributed simulation, formal transition systems, artificial life, and operational state-space geometry converges on a common problem: a world must remain identifiable while computation, observation, and participation continually change it. These literatures use the word state for several different things, and the differences matter for what a system is entitled to claim about itself. This paper separates four layers: rendered appearance, operative physical or simulated state, derived provenance and permission state, and append-only committed history. It argues that rendering and numerical evolution are disposable computations; that an observation becomes historical only when it is attested and constrains future admissibility; that current physical and provenance states should be certified folds over one authoritative event history; and that refusals and replay discrepancies must be appended rather than erased. Beyond restating this architecture, the paper’s organizing contribution is evidentiary rather than architectural: each claim recruited to support the four-layer model is graded into one of four epistemic classes—established result, legitimate structural analogy, emerging but unsettled research, or unsupported extrapolation—so that the strength of the synthesis is visible at the level of individual claims rather than asserted for the whole. The paper closes by naming, explicitly, the connections the surrounding literature does not yet license.

Contents

1	The layers question	2
2	A four-layer architecture	3
2.1	Rendering	3
2.2	Operative state	4
2.3	Provenance and permission state	4
2.4	Committed history	4
3	One history, several folds	4
4	Observation is not attestation	5
5	Admission, refusal, and repair	5
6	Checkpoints and replay divergence	5

7	Continuous fields without infinite history	6
8	Distributed worlds and causal history	6
9	Grading the evidence	7
9.1	Established results	7
9.2	Legitimate analogies	9
9.3	Emerging research	10
9.4	Grading methodology and a summary table	11
9.5	Unsupported connections	11
10	Precedents and how this architecture differs from them	12
11	A worked example	13
12	What the grading changes	13
13	Design rules for an implementation	14
14	Implications for implementers and reviewers	14
15	Open research questions	15
16	Conclusion	16
A	Formal sketches	17
A.1	Determinism of folds (supports the E-graded claim in Section 9.1)	18
A.2	Independence and commutativity (supports the A-graded claim in Section 8)	18
A.3	Non-monotonicity of refusal in M but monotonicity in H (supports the design rule “make refusal durable”)	18
A.4	Why bitwise, numerical, semantic, causal, and provenance recurrence are genuinely distinct grades	18

1 The layers question

A simulated world is easy to mistake for whatever appears on screen. Learned video systems intensify that mistake because they can generate visually convincing continuations without maintaining an explicit, queryable world state. Classical engines make the opposite simplification: they identify the world with the current values of objects, fields, inventories, and coordinates. Persistent and multiplayer systems expose a third requirement. A world must also remember which changes were admitted, which were refused, who acted, what evidence constrained the result, and how the present acquired its authority.

The research corpus supports a sharp initial distinction between a rendering latent and a state latent. Action-conditioned video generators condition future images on prior frames and actions; latent-dynamics systems learn iterable predictive structure that can support control or planning. But even an explicit state-transition model does not by itself preserve historical identity. A model can reproduce the same present state from incompatible pasts, silently replace one account with another, or restore a snapshot that looks correct while losing the commitments by which the state became legitimate.

rendered appearance $\neq$ operative state $\neq$ provenance state $\neq$ committed history

These are not four independent worlds. They are four roles within one architecture. Rendering presents a view. Operative state supports ongoing dynamics. Provenance state summarizes historically relevant relations such as ownership, attribution, access, and standing constraints. Committed history is the durable record from which authoritative states are derived.

The problem is not merely terminological housekeeping. Systems that conflate these layers make specific, costly mistakes. A generative video engine that treats visual continuity as world continuity will regenerate a previously visited location inconsistently, because nothing in its architecture distinguishes “looks the same” from “is the same object with the same history.” A classical multiplayer engine that treats the current value of M_t as the whole of what the world knows will have no principled way to answer a dispute about who owned an item three sessions ago, because the answer was never recorded as a fact separate from the current value it produced. A checkpoint-based system that silently substitutes a repaired reconstruction for a corrupted save file, without recording that a substitution occurred, will present a fabricated past as though it had always been the actual one, which is a stronger and more damaging error than simply losing data, because it removes the very evidence that would let anyone notice the loss.

This paper differs from a first pass at the same architecture in one deliberate respect. It does not simply assemble supporting citations under each layer and let the accumulation of names imply that the synthesis itself is established. Instead, Section 9 makes epistemic status a first-class object: every load-bearing claim is tagged as established, analogical, emerging, or unsupported, and the tags are used consistently in the sections that follow. A reader who wants only what the field has actually shown can read the established-result tags and skip the rest; a reader interested in where the architecture is speculative can find that marked as such rather than left to infer it from citation density.

2 A four-layer architecture

Layer	Primary object	Authority	Typical failure
Rendering	Observer-relative appearance x	None beyond presentation	Visual continuity mistaken for world continuity
Operative state	Fields, objects, variables M	Provisional dynamics	Snapshot treated as complete identity
Provenance state	Ownership, permissions, attributions Q	Certified historical projection	Context recomputed informally or lost
Committed history	Typed event DAG H	Identity and admissibility	Past silently patched after divergence

Table 1: The layers differ by function and authority; they need not use different storage systems.

2.1 Rendering

Rendering computes what a particular observer can presently receive. It may depend on viewpoint, visibility, level of detail, uncertainty, access rights, and presentation medium. It is intentionally

disposable: repeated render calls should not enlarge history merely because more observers looked. Rendering can be cached, approximated, regenerated, or degraded without changing what the world has committed to.

$$x_t = R(M_t, Q_t, \omega_t)$$

The observer parameter ω_t includes location, sensory channel, credentials, and other view conditions. The renderer reads certified current folds rather than scanning the complete event history at every frame.

2.2 Operative state

The operative state M_t contains what the dynamics need now: positions, velocities, fields, inventories, health values, object relations, and other variables used by transition rules. Between commitments it may evolve through many provisional solver steps. These steps matter computationally but need not each become part of historical identity.

$$\tilde{M}(t + dt) = F_{dt}(\tilde{M}(t); \theta, \xi)$$

Here θ identifies rules or solver versions and ξ records controlled stochastic input. The tilde marks a provisional state that can be recomputed or refined before any semantic commitment is made.

2.3 Provenance and permission state

The provenance fold Q_t carries relations that affect interpretation and admissibility without necessarily changing immediate physics: ownership, authorship, repair lineage, warrants, access rules, sanctions, unresolved claims, and scars understood as attributions. Keeping Q separate prevents physical state from becoming an unstructured warehouse for every historical relation while avoiding an unbounded history scan during ordinary operation.

2.4 Committed history

The authoritative history H is an append-only collection of typed events. In a single-authority system it may be a sequence. In multiplayer or distributed settings it is more naturally a directed acyclic graph whose edges record causal dependence. A committed event states not merely that a value changed, but that a candidate passed or failed an admissibility boundary under identifiable evidence and rules.

3 One history, several folds

The cleanest reconciliation is to treat M and Q as certified caches produced by different folds over the same history. This makes history authoritative without requiring every frame or numerical microstep to replay the entire past.

$$M_t = \text{fold}(\text{commit}_{\text{phys}}, M_0, H_t), \quad Q_t = \text{fold}(\text{commit}_{\text{prov}}, Q_0, H_t)$$

A physical commit function applies events that alter operative dynamics. A provenance commit function applies events that alter standing, attribution, access, or interpretation. Other folds may be added for accounting, narrative summaries, scientific measurement, or observer-specific indexes.

The world is therefore not simply a snapshot paired with a log. It is the authoritative log together with a family of certified folds and checkpoints.

Resolution. History is authoritative at commitment boundaries. State is authoritative between them only as the current certified or provisional result of admissible evolution.

4 Observation is not attestation

Observation must not be used as a loose synonym for commitment. If every view were an event, history would grow with observer count and rendering would acquire unintended causal power. Merely looking is a disposable computation. An observation becomes historical only when it is registered as evidence that constrains the admissible set.

looking: $R(M_t, Q_t, \omega) \rightarrow x$ [no commit]

attesting: $(x, \text{witness}, \text{rule}) \rightarrow E_{t+1} \rightarrow \text{narrower admissible set}$

This distinction aligns simulation engineering with an epistemic account of evidence. Sensors may sample continually while only selected, authenticated, or threshold-crossing measurements become attestations. A screenshot need not alter a world; a signed measurement used to authorize a repair can. The decisive property is not perceptual occurrence but constraining work.

5 Admission, refusal, and repair

A history containing only successful transitions is incomplete. Failed proposals disclose boundaries, attacks, conflicts, and changing expectations. The event vocabulary should therefore include at least proposed, admitted, refused, repaired, and replay-discrepant outcomes. A refusal is not a physical transition, but it is still an event about the relation between an attempted action and the world's constraints.

candidate $c \rightarrow \text{gate } A(c \mid M_t, Q_t, H_t) \rightarrow \{\text{admit, refuse, repair}\}$

The physical fold normally ignores a refusal because no admitted physical change occurred. The provenance fold may record it because an attempted intrusion, invalid transfer, or prohibited repair can alter permissions, suspicion, audit status, or future admissibility. Repair should likewise remain visible: replacing a failed proposal with a corrected one must not erase the distinction between what was first attempted and what was finally admitted.

6 Checkpoints and replay divergence

Checkpoints are necessary for large worlds, but they must be treated as accelerators rather than rival authorities. A checkpoint binds a folded state to a history frontier, ruleset, solver identity, seed policy, and verification hash. It permits efficient recurrence while preserving the claim that the state is certified by a particular history.

$C_k = (\text{frontier}_k, M_k, Q_k, \text{versions}_k, \text{seeds}_k, \text{hash}_k)$

Exact bitwise replay may fail because of nondeterministic solvers, floating-point differences, hardware changes, missing dependencies, or corrupted artifacts. The framework's policy is simple:

do not patch the earlier checkpoint or rewrite its event. Append a replay-discrepancy event containing the expected and obtained hashes, environment description, and resulting disposition. Later repair may create a new certified checkpoint while the failed recurrence remains part of provenance.

This policy separates epistemic immutability from operational rigidity. The system may improve its solver, restore service, or adopt a better reconstruction. What it may not do is present the improved result as though it had always been the original result.

7 Continuous fields without infinite history

A history-primary account need not record every timestep of a continuous field. The distinction is between numerical work and semantic commitment. Solver ticks approximate a trajectory; semantic events identify candidate changes; commits determine which changes have become identity-bearing facts.

$$\text{solver ticks} \rightarrow \text{semantic candidates} \rightarrow \text{committed events}$$

A field-level commit can record the initial or prior certified state, boundary conditions, forcing data, solver and operator versions, stochastic seeds, tolerance policy, interval, constraint checks, and resulting checkpoint hash. Intermediate values can be retained for diagnostics or discarded. What must survive is enough information to state what was asserted, under which procedure, and with what verification result.

Commit frequency is task-relative. A collision, transfer of ownership, public attestation, branching decision, or irreversible phase change may deserve a commit even when it occurs inside a much finer numerical integration. Conversely, thousands of stable substeps may remain provisional. The history records distinctions that matter to recurrence and accountability, not an indiscriminate transcript of computation.

8 Distributed worlds and causal history

Persistent multiplayer worlds cannot assume a single total order known everywhere at once. Interest management gives participants partial and sometimes stale projections; network delay creates concurrent proposals; rollback and reconciliation revise provisional states. The Chandy–Lamport distributed-snapshot result marks an important epistemic limit here, and it is worth being precise about exactly what it licenses before it is used.

The Chandy–Lamport algorithm proves that a set of processes communicating over reliable FIFO channels can construct a consistent global snapshot—a set of local states and in-transit channel contents that could have been the joint state of the system at some instant, even though no process ever observes such an instant directly and the snapshot need not correspond to any single moment of wall-clock time [3]. Every process records its own state on first receipt of a marker and reports the contents of each incoming channel between its own recording and the marker’s arrival on that channel; the union of these local records is guaranteed, by the FIFO and no-failure assumptions, to be a cut no message crosses backward through.

The relevance to persistent worlds is narrow and should not be overstated. What the algorithm establishes is that a distributed system can capture a globally consistent cut without a synchronized global clock and without halting computation—this is the established result, not an analogy. What it does not establish is anything about game engines, provenance, or attestation; those are the paper’s own extensions, not conclusions of the 1985 result. The legitimate transfer is structural: a persistent world’s committed history should likewise be a directed acyclic graph of causally related

commits rather than a single sequence stamped with wall-clock time, because concurrent, causally unrelated commits in different regions of a world have exactly the shape of Chandy–Lamport’s concurrent local states, and forcing them into one global order manufactures a false total order that the underlying causal structure does not support.

$$e_i \text{ independent of } e_j \implies \text{fold}(e_i \text{ then } e_j) = \text{fold}(e_j \text{ then } e_i)$$

This is the operational meaning of the residue-algebra principle that combination commutes only over causally independent commitments. When two actions compete for one object, depend on the same scarce resource, or alter each other’s permissions, the system must order, refuse, branch, or repair them. It cannot declare them independent for convenience. Independence must be a property the gate A can certify from Q_t and the causal frontiers named by each candidate event, not a default assumed in the absence of evidence to the contrary.

9 Grading the evidence

The preceding sections state an architecture. This section is the paper’s central methodological move: separating what that architecture can lean on from what it merely borrows vocabulary from. Four grades are used throughout, applied at the level of individual claims rather than whole literatures, because a single paper can contain a rigorously established theorem, a suggestive but untested analogy, and a piece of active controversy in adjacent sections.

Established (E).

A formally proved result or a widely replicated empirical finding, usable as a load-bearing premise.

Analogy (A).

A structural correspondence between a result in one domain and a claim in another, useful for intuition and design but not itself evidence that the target domain behaves as the source domain does.

Emerging (M).

An active research area with real results but unsettled scope, contested benchmarks, or a small number of demonstration systems rather than a mature theory.

Unsupported (U).

A connection that sounds plausible, that this paper’s own argument might tempt a reader to accept, but that no cited work actually demonstrates. Naming these is as important as naming the established results, because a synthesis paper’s main risk is exactly this kind of unmarked slide from analogy to claim.

9.1 Established results

State-machine replication (E). Deterministic operations applied in the same order at every replica produce identical replica states, given agreement on the order [16]. This is the foundational justification for treating M_t and Q_t as deterministic folds over H_t : if the fold functions are deterministic and every replica folds the same prefix of H in the same order, replicas converge. The result says nothing about provenance, attestation, or refusal; it only guarantees that agreement on a log implies agreement on derived state, which is exactly the load this paper puts on it and no more.

Virtual Time and optimistic distributed simulation (E). Jefferson’s Time Warp mechanism shows that a distributed simulation can proceed speculatively, using rollback and antimessages to correct causality violations discovered after the fact, with fossil collection reclaiming state that can no longer be rolled back to [11]. This is an established mechanism for exactly the operational problem Section 6 names: a provisional computation can run ahead of certainty and be corrected without having to pause the world. The paper’s replay-discrepancy policy (append, do not patch) is compatible with Time Warp’s rollback but is not identical to it; Time Warp’s antimessages retract a provisional computation before it is ever treated as committed, whereas this paper’s discrepancy events concern computations already treated as certified that later fail to reproduce. The two mechanisms operate on different sides of the commitment boundary, and treating them as the same mechanism would be an unmarked slide from established result to unexamined restatement.

Chandy–Lamport distributed snapshots (E). As detailed in Section 8, the algorithm is a proved result about consistent global cuts under FIFO channels and no failures. It is established exactly to the extent stated there, and the extension to game-world commit DAGs is flagged separately in Section 9.2 as analogy, not restated as established.

Structural operational semantics (E). Defining a transition relation by inference rules over syntactic configurations, so that admissible transitions are exactly those derivable from the rules, is a mature and standard technique in programming-language theory [15]. The gate function $A(c \mid M_t, Q_t, H_t) \rightarrow \{\text{admit}, \text{refuse}, \text{repair}\}$ in Section 5 is modeled on this idea: admissibility is derivability under a rule set applied to the current certified state, not an ad hoc check. What SOS does not supply is the three-way outcome (repair as a distinct category from admit/refuse) or any treatment of provenance; those are this paper’s additions, appropriately marked as extensions of an established framework rather than as themselves established.

POMDPs (E). Partially observable Markov decision processes formalize state-dependent action sets, belief states over hidden state, and noninvertible observation kernels [13]. This supplies precise vocabulary for why $R(M_t, Q_t, \omega_t)$ is many-to-one: an observer’s belief about M_t is a distribution induced by a noninvertible channel, and no single observation recovers M_t exactly. POMDP theory says nothing about committed history or attestation; it is imported here purely for its treatment of the observation channel.

Computational mechanics (E). Causal states, defined as equivalence classes of histories that induce the same conditional distribution over futures, are a rigorously developed formal object with proven minimality and uniqueness properties relative to a stochastic process [19, 5]. This gives the cleanest available formal gloss on M_t : it is the minimal sufficient statistic of $H_{\leq t}$ for predicting future admissible transitions, not the whole history and not an arbitrary compression of it. This is a genuinely established formal target, though no cited work has actually constructed causal states for a persistent-world commit history; the claim that M_t is (rather than should be modeled as) a causal state is graded as analogy in Section 9.2, not established.

Viability and reachability theory (E). Viability theory characterizes the set of states from which a trajectory can be kept within a constraint set indefinitely under some admissible control [1]; reachability theory characterizes which states are attainable under controlled dynamics from a given start. Both are established mathematical theories with worked machinery (viability kernels, capture basins). Their extension to a combined (M, Q) state space, so that provenance and institutional constraints count as part of the viability kernel, is listed as a research question in Section 15, not claimed as done.

Event structures (E). Winskel-style event structures formalize configurations that grow monotonically while conflict relations permanently exclude alternative continuations, giving irreversibility a precise home in concurrency theory. This is established and maps closely onto the append-only, conflict-excludes-alternatives behavior wanted for H ; the paper treats this as the nearest exist-

ing formal skeleton for H without claiming that H as described here has been proven to satisfy event-structure axioms in any implemented system.

Logical and vector clocks (E). Lamport’s original logical-clock construction and its later vector-clock extensions give a proved method for deriving a causal partial order (the happened-before relation) from message-passing alone, without a synchronized wall clock [12]. This is the mechanism that makes “recording a causal parent” in Section 13 an operational instruction rather than a metaphor: a commit’s parents can be computed directly from the vector-clock values attached to the events it causally depends on, and two commits are certified independent exactly when neither vector clock dominates the other. This is established machinery, prior to and presupposed by Chandy–Lamport’s own snapshot algorithm, which is itself built on the happened-before relation.

Conflict-free replicated data types (E). CRDT theory proves that certain data types (grow-only sets, counters, observed-remove sets, and others) admit merge operations that are commutative, associative, and idempotent, guaranteeing that replicas converge to the same state regardless of the order in which they receive updates, without any coordination protocol [17]. This is a directly relevant established result for Section 8’s commutativity condition: rather than treating $e_i \perp e_j \implies \text{fold}(e_i, e_j) = \text{fold}(e_j, e_i)$ as a bare postulate, the CRDT literature supplies a proof technique (semilattice-structured merge) for exactly which fold functions can satisfy it, and a matching negative result: operations that are not naturally expressible as a join over a semilattice (most contested-resource transfers, for instance) cannot be made CRDT-commutative without changing their semantics, which is the formal reason Section 5’s gate must refuse or serialize contested candidates rather than trying to merge them.

Eventual consistency and the CAP theorem (E). Brewer’s conjecture and its later formal proof establish that a distributed data store cannot simultaneously guarantee strong consistency, availability, and partition tolerance; a system must trade off which two it prioritizes during a network partition [10]. This is established and directly constrains what “certified fold” can mean in a genuinely distributed, partition-tolerant persistent world: M_t and Q_t as read by a partitioned region may be a stale but internally consistent fold over a strict prefix of H , not the fold over the full current H , and the architecture must therefore specify (as Section 6’s checkpoint frontier does) exactly which prefix a given certified state corresponds to rather than asserting a single global “current” state exists at every partition simultaneously.

9.2 Legitimate analogies

Ceptre as a close formal analogue (A). Ceptre models world state as a multiset of linear-logic resources consumed and produced by transitions, with a separate marking for permanent facts, and its execution traces carry causal-dependency structure [14]. This is the single closest existing formal system to the four-layer architecture: the multiset/permanent-fact split resembles M versus Q , and trace causal-dependency resembles the commit DAG of Section 8. It is graded as analogy rather than established precedent for this paper’s architecture because Ceptre was designed for authored interactive narrative generation, not for distributed multiplayer persistence, replay verification, or attestation; the resemblance is real and load-bearing for design intuition, but no result in the Ceptre literature demonstrates the checkpoint, replay-discrepancy, or attestation machinery this paper adds.

Chandy–Lamport applied to game-world commits (A). As flagged in Section 8, the transfer from “consistent cuts over communicating sequential processes” to “consistent cuts over causally related world-state commits” is an analogy this paper is making, not a theorem anyone has proved about game engines. The correspondence is structurally tight (both are partial orders of events related by message/dependency passing, both refuse a global total order, both admit multiple valid

linearizations), which is why the analogy is graded A rather than U; but it is an analogy the paper is proposing, and a reader should not come away thinking the 1985 result was already stated in these terms.

Computational mechanics’ causal states as a model for M_t (A). As noted above, treating M_t as literally a causal state in Crutchfield and Shalizi’s sense is an appealing target because it would give M_t a provably minimal and canonical form. No cited work constructs this for an event-sourced world history, so the correspondence remains an analogy pointing toward a research program (see Section 15, item 1) rather than a completed identification.

Artificial life and social simulation as epistemic instruments (A). Systems such as Schelling’s segregation model, Boids, Sugarscape, Tierra, Avida, and more recent large-scale multi-agent environments demonstrate that local per-agent rules and state can generate persistent, unplanned global organization, and that running such simulations can disclose consequences of a rule set that are not transparent from the rules alone. This supports using computational worlds as instruments for discovering emergent structure. It is an analogy to the four-layer architecture rather than a direct precedent, because the ALife literature is centered on emergence and pattern formation, generally without the provenance, attestation, or refusal machinery this paper adds; the layers question is largely orthogonal to what ALife research has been asking.

Quantum and categorical formalisms as comparative structural language (A). Generalized probabilistic theories, quantum instruments, categorical quantum mechanics, monotone information metrics, and Markov categories are rigorously developed languages for states, transformations, interventions, distinguishability, and composition [2, 4, 6]. Markov categories in particular give a clean categorical semantics for stochastic maps and conditional independence that could, in principle, be used to formalize the fold functions of Section 3 and the independence condition of Section 8. None of this literature establishes, or attempts to establish, a quantum-mechanical account of multiplayer game engines or learned world models; its role here is comparative, illustrating that rigorous operational theories of state and transformation exist and can be borrowed from, not that the target domain is quantum in any physical sense. This qualification is repeated because it is precisely the kind of claim most likely to be over-read.

9.3 Emerging research

Learned latent-dynamics models (M). Systems such as World Models, PlaNet, Dreamer, and MuZero learn an iterable latent state that supports prediction, planning, or policy learning from pixels or other high-dimensional observations [7, 8, 9, 18]. These are genuine, reproducible research results with real benchmark performance, which is why they are graded emerging rather than unsupported. But the field is unsettled in exactly the respect this paper cares about: none of these systems maintain an explicit, append-only, auditable event history of the kind Section 3 requires of H . Their latent state is a compression optimized for prediction accuracy, not for provenance or admissibility, and whether a causal-state-like object can be extracted from such a latent without retraining the system around an explicit history remains open.

Action-conditioned video generation and persistence mechanisms (M). Neural game-engine-style video generators demonstrate that convincing visual continuation can be produced by conditioning on prior frames and actions, without an explicit persistent world state. More recent systems add memory mechanisms, retained poses, timestamps, or geometric context specifically to address the failure mode where the same location, revisited, is regenerated inconsistently. This is active, fast-moving work, and the direction of travel (adding explicit memory because pure visual continuation does not preserve identity) is independent confirmation, from a different research community, of this paper’s core claim that persistence must be engineered rather than expected to emerge

from rendering quality. It remains emerging because the memory mechanisms in question are heterogeneous, not yet standardized, and generally still lack the typed-event, admit/refuse/repair vocabulary this paper argues is necessary; a system can retain more visual context without thereby having a committed history in the sense of Section 2.

MMO and large-scale persistence engineering (M). Practitioner and applied-systems literature on massively multiplayer persistence distinguishes live state management from durable persistence and documents that different consistency classes impose different logging and replay obligations. This is real engineering knowledge but is graded emerging rather than established in the formal sense used elsewhere in this section, because it is largely proprietary, heterogeneous across systems, and reported through practitioner accounts and postmortems rather than through a unified, peer-reviewed theory with proofs comparable to state-machine replication or Chandy–Lamport.

9.4 Grading methodology and a summary table

The grade assigned to each claim in Sections 9.1–9.3 follows three questions, applied in order. First, is there a proof or a widely replicated, quantitatively reported empirical result behind the claim, as opposed to a plausible mechanism or a single demonstration? If yes, and if the claim as stated in this paper does not extend the result beyond what was proved or measured, the grade is established. Second, if the claim extends a result to a new domain by structural resemblance, is the resemblance one the original authors could recognize as their result’s actual content (the same partial order, the same commutativity condition, the same minimality property), rather than a resemblance that depends on treating a technical term as if it carried its everyday connotations across domains? If the resemblance survives that check, the grade is analogy; the quantum-formalism case in Section 9.2 is graded analogy precisely because the mathematical resemblance (operational theories of state and transformation) survives the check even though the physical connotation of “quantum” does not transfer. Third, if the claim concerns an active research area with real systems and reported results but no settled theory of the specific property this paper needs from it (an explicit, auditable, provenance-preserving event history), the grade is emerging. Any claim that fails all three tests, meaning it neither is proved, nor survives the resemblance check, nor is backed by an active but unsettled research program, but is nonetheless the kind of claim the paper’s own narrative momentum would tempt a reader to accept, is explicitly listed in Section 9.5 as unsupported rather than left unstated, since leaving it unstated is exactly how an unsupported connection survives into a reader’s summary of the paper.

9.5 Unsupported connections

Naming what the argument does not license is not a formality; it is the check against the paper’s own rhetorical momentum. Four connections are explicitly flagged as unsupported.

“Persistent worlds are quantum systems” (U). Nothing in the cited quantum-formalism literature establishes, models, or predicts anything about the behavior of game engines, multiplayer servers, or learned world models. The comparative use of quantum-informational language in Section 9.2 must not be read as a claim that physical or simulated game worlds instantiate quantum mechanics, exhibit quantum behavior, or are usefully modeled as physics rather than as formally analogous mathematics. Any reading of this paper that extracts a quantum theory of game worlds is a misreading.

“The four-layer synthesis is already established in the literature” (U). No located work combines persistent-world engineering, learned world models, provenance-sensitive rendering, append-only refusal-aware history, and operational geometry into the specific four-layer/fold framework

presented here. The individual components (Section 9.1) are established; their combination into this architecture is the paper’s own construction and should be read as a proposal to be tested, not as a literature review of an existing consensus.

”Time Warp rollback and this paper’s replay-discrepancy policy are the same mechanism” (U). As noted in Section 9.1, these operate on different sides of the commitment boundary. Describing them as interchangeable, or claiming that Time Warp’s proofs of correctness transfer directly to the replay-discrepancy policy, would overstate what either result supports.

”A learned world model’s latent state already satisfies the causal-state or provenance requirements of this architecture” (U). Section 9.3 notes that current latent-dynamics systems are unsettled precisely on this point. No cited system has been shown to maintain a latent that is simultaneously a minimal sufficient statistic in the computational-mechanics sense and an auditable record satisfying the admit/refuse/repair vocabulary of Section 5. Claiming that either property already holds of existing learned world models would be an unsupported extrapolation from what those systems have actually been shown to do.

10 Precedents and how this architecture differs from them

Three existing software patterns resemble the committed-history layer closely enough that the differences need to be stated precisely rather than left to be inferred.

Event sourcing (A). The event-sourcing pattern in application architecture already treats the authoritative record as an append-only sequence of domain events, with current state recovered by replaying or folding that sequence, and snapshots used purely as a performance optimization rather than a rival source of truth. This is close enough to Section 3’s fold formalism that it is graded as analogy bordering on direct precedent for the single-writer case. What event sourcing as ordinarily practiced does not supply is a distinguished refusal event type treated as historically significant (a rejected command is typically simply not persisted, rather than persisted as a first-class refused event), a causal DAG rather than a single sequence for the multi-writer case, or any attestation/observation distinction, since application events are generated by trusted internal logic rather than external, possibly adversarial, witnesses.

Version control systems (A). Git’s commit graph is a directed acyclic graph of immutable, content-addressed snapshots with explicit parent pointers, and its willingness to represent divergent histories (branches) that later merge is structurally close to the causal-frontier bookkeeping in Section 8. The disanalogy is that Git’s merges are typically resolved by a human or a three-way merge heuristic operating outside the formal model, whereas Section 5’s gate function is meant to be a first-class, auditable part of the system itself, evaluated automatically against M_t , Q_t , and H_t rather than delegated to an external actor at merge time.

Blockchains and distributed ledgers (A, with a specific disanalogy). Public blockchains share the append-only, tamper-evident, causally-ordered-block structure this paper wants for H , and are worth citing precisely because the resemblance invites a specific error: assuming that the four-layer architecture requires, or is best served by, the proof-of-work or proof-of-stake consensus machinery blockchains use to establish agreement among mutually distrusting, permissionless participants. Nothing in this paper’s architecture requires Sybil resistance or a native token; a single-authority world, or a permissioned multi-authority world with a fixed participant set, needs an append-only, causally-ordered, typed-event history, but does not need the specific costly-consensus mechanism blockchains add to solve a harder and different problem (agreement among untrusted, unknown parties). Conflating the two is a common and specifically named error in the surrounding discourse on persistent-world economies, and is one instance of the general caution urged in Section 9.5:

structural resemblance is not license to import an entire adjacent mechanism wholesale.

11 A worked example

The layers and the grading can be made concrete with a single running example: transferring ownership of an object between two players whose regions of a world are handled by different simulation authorities and who are not, at the moment of transfer, causally synchronized.

Player p_1 proposes a transfer of object o to player p_2 . The candidate event is

$$c = (\text{transfer_request}, o, p_1, p_2, \text{vc}(p_1))$$

where $\text{vc}(p_1)$ is p_1 's current vector clock (Section 9.1, logical and vector clocks), attached so that the causal frontier of the request is recoverable later regardless of when it is actually processed. The gate evaluates $A(c \mid M_t, Q_t, H_t)$ against two conditions: whether M_t shows p_1 currently possessing o , and whether Q_t shows no standing encumbrance (a lien, an active dispute, a sanction) on o or on p_1 's transfer rights. Three outcomes are possible.

Admit. If both conditions hold and no concurrent conflicting candidate (a second transfer request for the same o , or a repossession claim) is causally concurrent with c under the vector-clock ordering, the transfer is admitted: an event $E_{t+1} = (\text{transferred}, o, p_1, p_2, \text{vc}(p_1), \text{hash})$ is appended to H , $\text{commit}_{\text{phys}}$ updates M_{t+1} to show p_2 possessing o , and $\text{commit}_{\text{prov}}$ updates Q_{t+1} to record the new ownership lineage.

Refuse. If Q_t shows an encumbrance, the gate refuses. The refusal is still appended to H as $(\text{refused}, c, \text{reason})$, even though $\text{commit}_{\text{phys}}$ leaves M untouched, because $\text{commit}_{\text{prov}}$ may need it: a pattern of refused transfer attempts against the same object is exactly the kind of provenance-relevant fact (Section 5) that should be visible to later audits even though no physical change occurred.

Repair. If a second, causally concurrent transfer request c' for the same o arrives from a different region before c is folded there, the CRDT result cited in Section 9.1 is directly applicable and directly limiting: ownership transfer of a single, non-divisible object is not a semilattice-mergeable operation, so the two concurrent candidates cannot both be admitted and cannot be silently merged. The gate must serialize them by some deterministic rule (for instance, the request with the causally later timestamp under a agreed tie-breaking total order loses), admit one, and append a **repaired** event referencing the other candidate's identifier, so that the losing player's client, when it later receives the causal frontier past the conflict, can reconstruct why its own locally-admitted-looking transfer was in fact superseded rather than silently vanished.

This example is deliberately small enough to make the grading concrete: the vector-clock frontier attached to c is established machinery (Section 9.1); the decision to serialize rather than merge concurrent transfers follows from an established negative result about which operations CRDTs can make commutative, not from an assumption; and the specific typed-event vocabulary (**transfer_request**, **transferred**, **refused**, **repaired**) is this paper's own proposal, not drawn from a cited standard, and should be read with the same weight as the other unmarked-as-established design choices in Section 13.

12 What the grading changes

Grading the evidence has a direct effect on how strongly Section 2's design rules should be read. A design rule resting on an established result (never repair the past in place, which follows from treating H as an event-structure-like object together with the epistemic-immutability discipline

of Section 6) can be stated as a requirement. A design rule resting on an analogy (organize concurrent commits as a Chandy–Lamport-style DAG) should be stated as a recommendation whose justification is structural resemblance, not as a theorem inherited from distributed systems. A design rule motivated by emerging work (add explicit memory to a learned world model to approximate provenance) should be stated as a hypothesis to be tested against the specific system in question, because the emerging literature has not yet converged on how such memory should be structured.

This has a further consequence for the research-questions list in Section 15: several of the “open questions” from an architecture-only presentation of this material are, once graded, better described as the analogy-to-established-result gap named in Section 9.2, rather than as free-floating unknowns. Making that gap explicit is more useful than listing it as an undifferentiated open question, because it tells a reader exactly which existing formal machinery (computational mechanics, viability theory, event structures) would need to be extended, and in what specific respect, to close it.

13 Design rules for an implementation

- Keep rendering pure (E-grounded). A render call may read certified folds but should not write history merely because a view was produced.
- Commit attestations, not perceptions (E-grounded, via the observation/attestation distinction of Section 4). Evidence enters history only when an identified procedure uses it to constrain admissibility.
- Use typed events (extension of SOS, marked as this paper’s addition). Distinguish proposals, admissions, refusals, repairs, attestations, checkpoints, and replay discrepancies.
- Derive current authority (E-grounded, via state-machine replication). Physical and provenance states are folds or certified checkpoints tied to explicit history frontiers.
- Make refusal durable (this paper’s addition, not established elsewhere). A failed proposal may leave physical state unchanged while still affecting audit and permission folds.
- Never repair the past in place (E-grounded, via event structures and epistemic immutability). A corrected reconstruction or checkpoint is appended with its relation to the failed one.
- Record causal parents (A-grounded, via the Chandy–Lamport analogy of Section 8). Distributed commits name their dependencies so independent events can commute and conflicting ones cannot.
- Separate recurrence levels (this paper’s addition). Bitwise, numerical, semantic, causal, and provenance-preserving recurrence must not be collapsed into one replay claim.

14 Implications for implementers and reviewers

Two audiences read a paper like this for different reasons, and the grading in Section 9 is meant to serve both without forcing either to do the other’s work.

An implementer building a persistent world can treat the established-grade claims (Section 9.1) as load-bearing: a system that violates state-machine determinism, that ignores the CRDT commutativity boundary when merging concurrent updates, or that assumes a single global total order

across a genuinely partitioned deployment, is not making a design tradeoff but reproducing a known failure mode with a known fix in the literature. The analogy-grade claims (Section 9.2) should inform design intuition and naming (calling the commit graph a “Chandy–Lamport-style DAG” is a legitimate and useful shorthand) without being treated as a source of correctness guarantees the implementer can cite in place of doing the CRDT or vector-clock analysis for their own specific merge operations. The emerging-grade claims (Section 9.3) should be read as an invitation to prototype cautiously, with the explicit expectation that the relevant literature may look different in two years, rather than as a stable foundation to build a production system’s core identity guarantees on.

A reviewer or a skeptical reader evaluating a synthesis paper of this kind has a narrower and more specific job: checking whether the unsupported-connections list (Section 9.5) is actually complete, meaning whether there are further places where the paper’s own rhetorical momentum extends an established result or a legitimate analogy past what it was shown to support. This paper’s own answer to that check is necessarily incomplete, since a paper cannot fully audit its own blind spots; the grading table (Table 2) is offered as the concrete surface on which such an audit can be conducted claim by claim, rather than as a claim that the audit has already been performed exhaustively.

One further implication concerns revision practice specifically. Where an earlier presentation of this architecture folded citations and design rules together under a single undifferentiated “established in the literature” framing, later criticism correctly identified this as a place where confidence had outrun evidence. The response adopted here is not to weaken the architecture’s claims uniformly, which would discard genuine established results alongside genuine speculation, but to differentiate them, so that the parts resting on Schneider, Chandy and Lamport, Lamport’s vector clocks, and Shapiro et al.’s CRDT theorem keep their full evidentiary weight while the four-layer combination itself, and the specific typed-event vocabulary proposed in Section 5, are correctly presented as this paper’s own construction, open to revision or rejection independent of whether the underlying established results continue to hold.

15 Open research questions

1. What is the smallest event vocabulary sufficient to reconstruct both operative and provenance folds without making every solver step historical? (Closing the analogy-to-established gap between Section 5’s typed events and a minimality proof comparable to computational mechanics’ causal states.)
2. Which admissibility predicates must be evaluated against current folds, and which require direct reference to a historical witness?
3. When may two events commute in a distributed history, and how can independence be certified rather than assumed, given that the Chandy–Lamport correspondence of Section 8 is an analogy rather than a proof for this specific setting?
4. How should replay equivalence be graded across bitwise, numerical, semantic, causal, and provenance-preserving recurrence, and can these grades be given the same kind of formal minimality treatment computational mechanics gives causal states?
5. Can a learned world model acquire persistent identity without an explicit event ledger, or does apparent persistence merely relocate the ledger into opaque memory? Section 9.3 suggests the honest current answer is that no cited system has settled this either way.

6. How should refusal affect future permissions without allowing adversarial attempts to inflate or poison the authoritative history?
7. Can viability kernels be defined over the combined folded state (M, Q) , making institutional and provenance constraints part of reachable future volume, extending viability theory (Section 9.1) beyond its established scope?
8. Which information metric best measures the loss incurred when a historically distinct world is collapsed into an extensionally identical snapshot?

16 Conclusion

The layers question is resolved by assigning different kinds of authority to different representations. Rendering governs appearance but creates no history. Operative state governs provisional evolution between commitment boundaries. Provenance and permission state summarize historically active relations. Committed history governs identity, auditability, and the legitimacy of recurrence. One event history can support many efficient folds, so historical authority need not imply continuous log scanning or the storage of every numerical microstep.

The decisive boundary is not between computation and storage but between provisional work and attested constraint. Looking is not an event; attesting is. A refusal can be historically real without producing a physical change. A failed replay can be repaired without rewriting what failed. A snapshot can accelerate reconstruction without becoming the source of truth.

But the architecture is only as trustworthy as the honesty of its citations, and a synthesis paper's characteristic failure is not a false premise but an unmarked slide from a real result to an unearned generalization of it. This paper has tried to make that slide visible rather than hide it: eleven results are established, five correspondences are flagged as analogies whose intuitive force should not be mistaken for proof, three research programs are acknowledged as active but unsettled, and four specific claims that the argument's own momentum might otherwise smuggle in are named and rejected. In this architecture, a world persists not because it renders the same scene again, but because it can state what was committed, what was refused, and why the present is entitled to count as a continuation of its past — and a paper about that architecture earns its own credibility the same way, by stating plainly which of its claims are owed to prior results and which are being proposed for the first time.

References

- [1] Aubin, J.-P. (1991). *Viability Theory*. Birkhäuser.
- [2] Barrett, J. (2007). Information processing in generalized probabilistic theories. *Physical Review A*, 75, 032304.
- [3] Chandy, K. M., & Lamport, L. (1985). Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1), 63–75.
- [4] Coecke, B., & Kissinger, A. (2017). *Picturing Quantum Processes*. Cambridge University Press.
- [5] Crutchfield, J. P., & Young, K. (1989). Inferring statistical complexity. *Physical Review Letters*, 63, 105–108.

- [6] Fritz, T. (2020). A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370, 107239.
- [7] Ha, D., & Schmidhuber, J. (2018). World Models. *arXiv:1803.10122*.
- [8] Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. *Proceedings of ICML*. *arXiv:1811.04551*.
- [9] Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to Control: Learning behaviors by latent imagination. *Proceedings of ICLR*. *arXiv:1912.01603*.
- [10] Gilbert, S., & Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
- [11] Jefferson, D. R. (1985). Virtual time. *ACM Transactions on Programming Languages and Systems*, 7(3), 404–425.
- [12] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
- [13] Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2), 99–134.
- [14] Martens, C. (2015). Ceptre: A language for modeling generative interactive systems. *Proceedings of AIIDE*.
- [15] Plotkin, G. D. (2004). A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60–61, 17–139.
- [16] Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach. *ACM Computing Surveys*, 22(4), 299–319.
- [17] Shapiro, M., Preguica, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types. *Proceedings of SSS 2011 (Stabilization, Safety, and Security of Distributed Systems)*, 386–400.
- [18] Schrittwieser, J., et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588, 604–609.
- [19] Shalizi, C. R., & Crutchfield, J. P. (2001). Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of Statistical Physics*, 104, 817–879.
- [20] Winsberg, E. (2010). *Science in the Age of Computer Simulation*. University of Chicago Press.

A Formal sketches

The following sketches are informal but precise enough to show where each design rule’s justification actually comes from. They are sketches, not full proofs; each is graded consistently with the section that motivates it.

A.1 Determinism of folds (supports the E-graded claim in Section 9.1)

Claim. If $\text{commit}_{\text{phys}}$ is a deterministic function of its two arguments and every replica applies it to the same ordered prefix of H , then all replicas compute the same M_t .

Sketch. By induction on the length of the prefix. The base case M_0 is given identically to all replicas. For the inductive step, suppose all replicas agree on M_k after folding the first k events, presented in the same order. Since $\text{commit}_{\text{phys}}$ is a deterministic function, applying it to the same M_k and the same $(k+1)$ -th event produces the same M_{k+1} at every replica. This is exactly Schneider’s state-machine-replication argument [16], restated for the fold notation of Section 3; no new proof content is claimed here beyond the restatement.

A.2 Independence and commutativity (supports the A-graded claim in Section 8)

Claim. If two events e_i, e_j are causally concurrent (neither’s vector clock dominates the other’s) and the fold’s action on each event, restricted to the state components each event touches, forms a join-semilattice update, then $\text{fold}(e_i \text{ then } e_j) = \text{fold}(e_j \text{ then } e_i)$.

Sketch. This is precisely the CRDT convergence theorem [17]: commutativity of merge for concurrently applied updates holds exactly when the update operation is a semilattice join, which is associative, commutative, and idempotent by definition of a semilattice. The claim is therefore not a new result; it restates the known necessary structural condition and makes explicit, as a corollary, why the object-transfer example of Section 11 cannot rely on it: exclusive possession transfer of a single object is not expressible as a join over any natural semilattice ordering on ownership, since possession is not itself monotonically increasing under merge, so the antecedent of the claim fails and the gate must fall back to serialization or refusal rather than merge.

A.3 Non-monotonicity of refusal in M but monotonicity in H (supports the design rule “make refusal durable”)

Observation. $H_{t+1} \supsetneq H_t$ always (append-only), but M_{t+1} need not differ from M_t when the appended event is a refusal. This is not a tension: H ’s monotonicity is a property of the record, while M ’s update is a property of one particular fold applied to that record. A refusal event is, by construction, in the kernel of $\text{commit}_{\text{phys}}$ (it maps to the identity update on M) while typically not being in the kernel of $\text{commit}_{\text{prov}}$. This is simply the definition of the two commit functions given in Section 3, stated here to make explicit that no additional axiom is needed to license refusal events; the architecture already accommodates them once $\text{commit}_{\text{phys}}$ and $\text{commit}_{\text{prov}}$ are allowed to be different functions of the same event.

A.4 Why bitwise, numerical, semantic, causal, and provenance recurrence are genuinely distinct grades

Bitwise recurrence requires identical output bits given identical input bits and code; it can fail from floating-point non-associativity alone under a changed instruction set, with no change to θ or ξ in Section 7’s notation. Numerical recurrence requires agreement within a stated tolerance and is the grade actually achievable across heterogeneous hardware. Semantic recurrence requires only that the same committed events be derivable, regardless of intermediate solver values, and is the grade Section 7 argues is usually sufficient. Causal recurrence requires only that the causal partial order of commits be reproduced, permitting different but causally-equivalent interleavings of independent events, per Section 8. Provenance recurrence requires only that the final Q_t be reachable, permitting different admitted/refused/repaired paths to the same standing relations. Each grade is strictly

weaker than the one before it in the sense that satisfying a stronger grade implies satisfying every weaker one, but not conversely; a system's replay guarantee should therefore always be stated as one specific grade rather than as an unqualified claim to have "reproduced the world."

Claim	Grade	What licenses the grade
State-machine replication $\Rightarrow$ deterministic folds converge	E	Proved (Schneider 1990); restated without extension
Virtual Time / rollback $\neq$ replay-discrepancy policy	E	Established mechanism; explicit disanalogy drawn
Chandy–Lamport consistent cuts exist without a global clock	E	Proved (Chandy & Lamport 1985)
Vector clocks derive a causal partial order from messages	E	Proved (Lamport 1978)
CRDT merges converge iff semilattice-structured	E	Proved (Shapiro et al. 2011)
CAP: cannot have strong consistency, availability, and partition tolerance together	E	Proved (Gilbert & Lynch 2002)
SOS: admissible transitions derivable from rules	E	Established framework (Plotkin)
POMDPs: observation channel is noninvertible	E	Established formalism
Causal states are minimal predictive sufficient statistics	E	Proved (Shalizi & Crutchfield 2001)
Viability/reachability kernels are well-defined constructs	E	Established theory (Aubin 1991)
Event structures formalize monotonic, conflict-excluding growth	E	Established (Winskel)
Chandy–Lamport cuts $\rightarrow$ game-world commit DAGs	A	Structural resemblance, not proved for this domain
Ceptre resource semantics $\rightarrow$ this paper’s M/Q split	A	Close formal resemblance, different design purpose
Causal states $\rightarrow M_t$ specifically	A	Resemblance is the target, not yet constructed
ALife/social simulation $\rightarrow$ epistemic instrument use	A	Supports method, not the layer architecture directly
Quantum/categorical formalisms $\rightarrow$ comparative language	A	Mathematical resemblance only, no physical claim
Event sourcing $\rightarrow$ committed-history layer	A	Close single-writer precedent, lacks refusal/DAG
Git commit graph $\rightarrow$ causal-frontier bookkeeping	A	Structural match, merge resolution left external
Blockchains $\rightarrow$ append-only ledger	A	Structure matches; consensus mechanism does not transfer
Learned latent-dynamics models (Dreamer, MuZero, etc.)	M	Real results, no explicit auditable history
Persistence mechanisms in generative video models	M	Active, fast-moving, heterogeneous, unstandardized
MMO persistence engineering practice	M	Real but largely proprietary, non-unified
Persistent worlds are quantum systems	U	No cited work supports a physical claim
Four-layer synthesis already exists in the literature	U	Components established; combination is this paper’s own
Time Warp and replay-discrepancy are the same mechanism	U	Operate on different sides of the commitment boundary
Learned latents already satisfy causal-state/provenance requirements	U	Unsettled per the emerging-grade entries above

Table 2: Every load-bearing claim in this paper, graded and justified in one place.